```
    xt = mnist_test.test_data[batch_ids].detach()
    xt =
prepare_images(xt,maxtrans=6,rot=30,noise=10).unsqueeze(dim=1)
    yt = mnist_test.test_labels[batch_ids].detach()
    preds = model(xt)
    pred_ind = torch.argmax(preds.detach(),dim=1)
    acc = (pred_ind == yt).sum().float() / batch_size
    return acc, xt, yt

acc2, xt2, yt2 = test_acc(agent)
print(acc2)
>>> 0.9460
```

We get nearly 95% accuracy at test time with the relational module after just 1,000 epochs of testing. Again, 1,000 epochs with a batch size of 300 is not enough to reach maximal accuracy. Maximal accuracy with any decent neural network on (unperturbed) MNIST should be around the 98–99% mark. But we're not going for maximum accuracy here; we're just making sure it works and that it performs better than a convolutional neural network with a similar number of parameters.

We used the following simple CNN as a baseline, which has 88,252 trainable parameters compared to the relational module's 85,228. The CNN actually has about 3,000 more parameters than our relational module, so it has a bit of an advantage.

Listing 10.6. Convolutional neural network baseline for MNIST

```
class CNN(torch.nn.Module):
    def __init__(self):
      super(CNN, self).__init__()
      self.conv1 = nn.Conv2d(1,10,kernel_size=(4,4))      1
      self.conv2 = nn.Conv2d(10,16,kernel_size=(4,4))
      self.conv3 = nn.Conv2d(16,24,kernel_size=(4,4))
      self.conv4 = nn.Conv2d(24,32,kernel_size=(4,4))
      self.maxpool1 = nn.MaxPool2d(kernel_size=(2,2))     2
      self.conv5 = nn.Conv2d(32,64,kernel_size=(4,4))
      self.lin1 = nn.Linear(256,128)
      self.out = nn.Linear(128,10)                        3
    def forward(self,x):
      x = self.conv1(x)
      x = nn.functional.relu(x)
      x = self.conv2(x)
      x = nn.functional.relu(x)
      x = self.maxpool1(x)
```